

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Vetores

Vetores

○ O QUE SÃO?

- São conjuntos de variáveis de **mesmo tipo**, acessadas por índices.
 - Em C, os índices começam em 0.
- Podem armazenar tipos primitivos, structs, vetores (matrizes), ponteiros e enums.
- A alocação de memória pode ser feita de forma **estática** ou **dinâmica**:
 - A **alocação estática** (tamanho definido) ocorre durante a compilação.
 - A **alocação dinâmica** (tamanho variável) acontece em tempo de execução, utilizando ponteiros e funções como **malloc**, **calloc**, **realloc** e **free**.
 - A escolha entre os dois métodos depende do tamanho do vetor e se o tamanho é conhecido.

- Sintaxe da alocação estática:

tipo_armazenado nome_do_vetor[tamanho].

- Sintaxe da alocação dinâmica (ponteiros): ← Veremos depois!!!

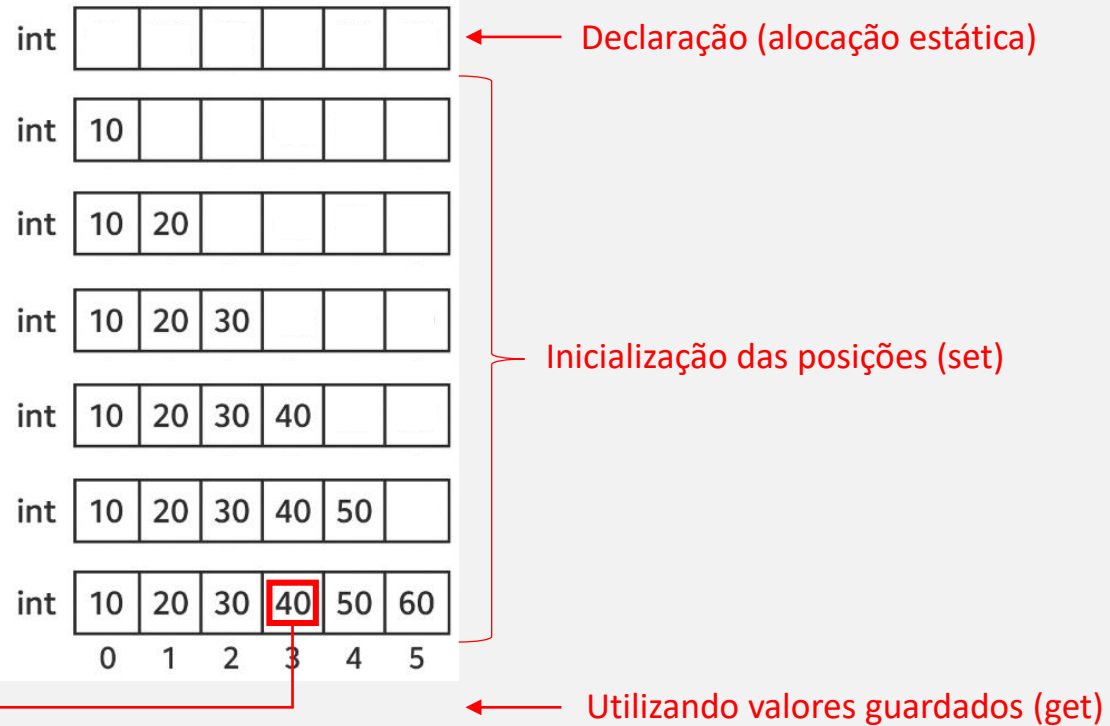
tipo_armazenado *nome_do_vetor = (**tipo_armazenado** *) malloc(tamanho * sizeof(**tipo_armazenado**));

Falaremos com detalhes
na aula de ponteiros!

Vetores

○ EXEMPLO

- `int vec[6];`
- `vec[0] = 10;`
- `vec[1] = 20;`
- `vec[2] = 30;`
- `vec[3] = 40;`
- `vec[4] = 50;`
- `vec[5] = 60;`
- `int num = vec[3];`



Vetores

○ EXEMPLO:

```
main.c  Alocação estática
1  #include <stdio.h>
2
3  int main()
4  {
5      // Alocação estática: espaço para até 100 inteiros
6      int vetor[100];
7      int tamanho, i;
8
9      printf("Digite o tamanho (máximo 100): ");
10     scanf("%d", &tamanho);
11
12     if (tamanho > 100 || tamanho < 1)
13     {
14         printf("Tamanho inválido!\n");
15         return 1;
16     }
17
18     // Preenchendo o vetor
19     for (i = 0; i < tamanho; i++)
20     {
21         printf("Digite o valor para vetor[%d]: ", i);
22         scanf("%d", &vetor[i]);
23     }
24
25     // Mostrando os valores armazenados
26     printf("\nConteúdo do vetor:\n");
27     for (i = 0; i < tamanho; i++)
28     {
29         printf("vetor[%d] = %d\n", i, vetor[i]);
30     }
31
32     return 0;
33 }
```

```
main.c  Alocação dinâmica
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int *vetor;
7      int tamanho, i;
8
9      printf("Digite o tamanho do vetor: ");
10     scanf("%d", &tamanho);
11
12     // Alocação dinâmica de memória
13     vetor = (int *)malloc(tamanho * sizeof(int));
14
15     // Verifica se a alocação deu certo
16     if (vetor == NULL)
17     {
18         printf("Erro ao alocar memória!\n");
19         return 1;
20     }
21
22     // Preenchendo o vetor
23     for (i = 0; i < tamanho; i++)
24     {
25         printf("Digite o valor para vetor[%d]: ", i);
26         scanf("%d", &vetor[i]);
27     }
28
29     // Mostrando os valores armazenados
30     printf("\nConteúdo do vetor:\n");
31     for (i = 0; i < tamanho; i++)
32     {
33         printf("vetor[%d] = %d\n", i, vetor[i]);
34     }
35
36     // Libera a memória alocada
37     free(vetor);
38
39     return 0;
40 }
```

Vetores

○ OPERADOR []

- Em C, o operador [] é usado para declarar um vetor e acessar elementos individuais de um vetor, seja para obter o valor (get) ou para modificar o valor (set).
- O operador funciona como um índice, onde o valor entre os colchetes indica a posição do elemento desejado dentro do vetor, começando por zero para o primeiro elemento.

○ Obter (Get):

- Para acessar o valor de um elemento em um vetor, utiliza-se o operador [] seguido do índice do elemento desejado.

○ Modificar (Set):

- Para modificar o valor de um elemento em um vetor, também se utiliza o operador [] com o índice desejado, mas desta vez no lado esquerdo de uma atribuição.

Vetores

○ INICIALIZAÇÃO

- Assim como os **tipos primitivos**, em C os vetores podem ser inicializados **no instante de suas declarações** ou **posteriormente**.

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int numeros[5]; // Declaração: vetor de 5 inteiros
6
7     // Atribuindo valores (set)
8     numeros[0] = 10;
9     numeros[1] = 20;
10    numeros[2] = 30;
11    numeros[3] = 40;
12    numeros[4] = 50;
13
14    // Acessando e exibindo valores (get)
15    printf("Conteúdo do vetor:\n");
16    printf("numeros[0] = %d\n", numeros[0]);
17    printf("numeros[1] = %d\n", numeros[1]);
18    printf("numeros[2] = %d\n", numeros[2]);
19    printf("numeros[3] = %d\n", numeros[3]);
20    printf("numeros[4] = %d\n", numeros[4]);
21
22    return 0;
23 }
```

Set: operador [] ao lado esquerdo do operador de atribuição =

Get: operador [] sem operador de atribuição

Exemplo de inicialização com atribuição posterior à declaração.

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     // Declaração e atribuição: vetor de
6     // 5 inteiros com valores já preenchidos
7     int numeros[5] = {10, 20, 30, 40, 50};
8
9     // Acessando e exibindo valores (get)
10    printf("Conteúdo do vetor:\n");
11    printf("numeros[0] = %d\n", numeros[0]);
12    printf("numeros[1] = %d\n", numeros[1]);
13    printf("numeros[2] = %d\n", numeros[2]);
14    printf("numeros[3] = %d\n", numeros[3]);
15    printf("numeros[4] = %d\n", numeros[4]);
16
17    return 0;
18 }
```

Exemplo de declaração já com atribuição.

Vetores

○ RAPIDEZ DE ACESSO

- A rapidez do acesso às posições dos vetores se deve ao fato de que o **endereço** de memória de cada elemento é **calculado** a partir do **endereço base** do vetor e do **índice** do elemento, levando em conta o tamanho em bytes do tipo de dado armazenado no vetor.
- Cálculo do endereço:
 - $\text{endereço_do_elemento} = \text{endereço_base} + (\text{índice} * \text{tamanho_do_tipo})$.
 - **endereço_base** é o endereço do primeiro elemento do vetor.
 - **índice** é a posição do elemento desejado no vetor (começando em 0).
 - **tamanho_do_tipo** é o tamanho em bytes do tipo de dado armazenado no vetor.
 - Ex: 4 bytes para **int**, 1 byte para **char**.
- Desta forma, se quisermos acessar a posição 500 de um vetor, o programa não incrementa de 0 a 500.
 - O acesso é direto pois o endereço de memória da posição 500 é calculado!

Vetores

○ CUIDADOS

- Pelo fato de a indexação de vetores em C iniciar pelo 0, um vetor de tamanho **N** terá posições de **0** a **(N - 1)**.
- Vetores alocados estaticamente, frequentemente têm tamanhos superiores ao necessário para garantir que atendam à necessidade do programa.
 - Nestes casos, a quantidade realmente utilizada deverá ser controlada manualmente.
- C não verifica a validade dos índices de um vetor.
 - O acesso a uma posição fora dos limites de um vetor no:
 - **get**: será retornado o que estiver armazenado ali naquela posição de memória, ou seja, é o que chamamos de “lixo de memória” por não fazer parte do programa.
 - **set**: pode sobrescrever o que antes estava ali naquela memória, comprometendo a confiabilidade do seu código.

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int numeros[20]; // Declaração: vetor de 20 inteiros
6
7     // Atribuindo valores (set) de 5 posições
8     int sz = 0; // Contador do tamanho utilizado
9     numeros[0] = 10;
10    sz++;
11
12    numeros[1] = 20;
13    sz++;
14
15    numeros[2] = 30;
16    sz++;
17
18    numeros[3] = 40;
19    sz++;
20
21    numeros[4] = 50;
22    sz++;
23
24    // Acessando e exibindo valores (get)
25    printf("Tamanho alocado: %d\n", 20);
26    printf("Tamanho utilizado: %d\n", sz);
27    printf("Conteúdo do vetor:\n");
28    for (int i = 0; i < sz; i++)
29    {
30        printf("numeros[%d] = %d\n", i, numeros[i]);
31    }
32
33    return 0;
34 }
```

Tamanho alocado: 20
Tamanho utilizado: 5
Conteúdo do vetor:
numeros[0] = 10
numeros[1] = 20
numeros[2] = 30
numeros[3] = 40
numeros[4] = 50

Vetores

○ VANTAGENS DA ALOCAÇÃO ESTÁTICA:

- **Simplicidade:** é mais simples de implementar e entender, pois o tamanho do vetor é definido no código e não requer funções de alocação dinâmica.
- **Velocidade de Acesso:** Acesso aos elementos do vetor é rápido e direto, pois a posição de cada elemento é conhecida no momento da compilação.
- **Sem sobrecarga de funções:** Não há necessidade de chamar funções como **malloc** ou **calloc** para alocar memória, o que pode otimizar o desempenho em alguns casos.
- **Segurança:** Evita erros comuns da alocação dinâmica, como *memory leak* ou acessos inválidos.

○ DESVANTAGENS DA ALOCAÇÃO ESTÁTICA :

- **Tamanho Fixo:** O tamanho do vetor deve ser conhecido no momento da compilação, o que pode ser limitante se o número de elementos variar durante a execução do programa.
- **Desperdício de memória:** Se o vetor for declarado com um tamanho maior do que o necessário, parte da memória ficará inutilizada. Se for declarado com tamanho insuficiente, pode ocorrer estouro de buffer.
- **Inflexibilidade:** Não é possível redimensionar o vetor dinamicamente após a sua criação, o que pode ser problemático para aplicações que requerem estruturas de dados adaptáveis.

Vetores

○ VANTAGENS DA ALOCAÇÃO DINÂMICA:

- **Flexibilidade:** Permite que o tamanho do vetor seja definido em tempo de execução, adaptando-se às necessidades do programa.
- **Reutilização de memória:** Blocos de memória liberados podem ser realocados para outros objetos ou estruturas, otimizando o uso dos recursos do sistema.
- **Criação de estruturas dinâmicas:** Ideal para estruturas de dados que podem crescer ou diminuir durante a execução, como listas encadeadas e árvores.
- **Evita desperdício de memória:** Aloca apenas o espaço necessário, reduzindo o desperdício de memória em comparação com a alocação estática com tamanho pré-definido.

○ DESVANTAGENS DA ALOCAÇÃO DINÂMICA:

- **Memory Leak:** Se a memória alocada dinamicamente não for liberada corretamente após o uso, pode levar a vazamentos de memória, diminuindo a quantidade de memória disponível para o sistema.
- **Gerenciamento manual:** O programador precisa gerenciar a alocação e liberação de memória manualmente, o que pode ser propenso a erros, como esquecer de liberar memória alocada.
- **Sobrecarga de desempenho:** A alocação e liberação dinâmica podem ser mais lentas do que a alocação estática, especialmente se feitas frequentemente.